Object Oriented Programming (CSEG1044)
Unit 01 – Lecture 03 Notes
I/O + Command Line + Types + Operators

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **3**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Console I/O: `Scanner` (most common for beginners) . . . 3
7.2 Console I/O: `BufferedReader` (fast, line-oriented) . . . 4
7.3 Output formatting . . . 4
7.4 Command line arguments . . . 4
7.5 Data types, operators, precedence, casting . . . 4
7.5.1 Primitive data types (quick) . . . 4
7.5.2 Operator pitfalls you must know . . . 4
7.5.3 Casting . . . 5

8 Running Project Connection **5**

9 Common Mistakes and Confusions **5**

10 Worked Example: Args + Scanner + Formatting **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **7**

14 Further Study **7**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Introduction to OOPs and Java Fundamentals
- **Main new ideas:** Console I/O: `Scanner` (most common for beginners), Console I/O: `BufferedReader` (fast, line-oriented), Output formatting
- **Course thread:** Use I/O + Command Line + Types + Operators to make the first text-based version of Campus Resource Manager easier to reason about before the full object model arrives.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

2 Lecture Overview

This lecture focuses on the day-to-day basics you need to write useful Java programs:

- taking input (keyboard / terminal),
- printing output in a readable way,
- using command line arguments,
- understanding primitive data types and operators (and common pitfalls).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Program structure**: You already know where execution starts in Java; this lecture fills that structure with input and expressions. Main reference: `Unit 01 / Lecture 02 slides/notes`.
- **Basic data handling**: The C prerequisite gives the intuition for values and operators; Java adds stronger typing rules. Main reference: Programming in C prerequisite.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Read input using `Scanner` and print formatted output.
- Use command line arguments (`args`) for simple programs.
- Apply primitive types, operators, precedence, and casting correctly.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Console I/O: `Scanner` (most common for beginners) → Console I/O: `BufferedReader` (fast, line-oriented) → Output formatting → Command line arguments → Data types, operators, precedence, casting.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Token vs line**: token = one word/number; line = full line of text.
- **Formatting**: printing numbers/text with a chosen format (decimal places, alignment).
- **Primitive types**: built-in value types like `int`, `double`, `char`, `boolean`.
- **Type casting**: converting a value from one type to another (widening/narrowing).
- **Precedence**: order in which operators are evaluated.

7 Detailed Notes

7.1 Console I/O: `Scanner` (most common for beginners)

`Scanner` reads tokens from an input source like `System.in`.

- Use `nextInt()`, `nextDouble()`, `next()` (word), `nextLine()` (whole line).
- **Common pitfall:** mixing `nextInt()` and `nextLine()`.

  After reading a number, the newline remains in the buffer; the next `nextLine()` may return an empty string.

Fix: call an extra `nextLine()` after reading the number.

7.2 Console I/O: `BufferedReader` (fast, line-oriented)

`BufferedReader` reads full lines as `String`, and you parse manually:

- Good when you want complete control or faster I/O.
- Slightly more code than `Scanner` (because you must parse numbers using `Integer.parseInt`, etc.).

7.3 Output formatting

Use:

- `System.out.println(...)` for simple output.
- `System.out.printf("x = %d, y = %.2f%n", x, y);` for formatted output.

Common format specifiers:

- `%d` integer, `%f` floating-point, `%s` string, `%n` newline.

7.4 Command line arguments

When you run a Java program, anything after the class name becomes a string in `args`.

- Example: `java Sum 10 20 30`
- Then: `args[0] = "10"`, `args[1] = "20"`, etc.

You often parse numbers using:

- `Integer.parseInt(args[i])`
- `Double.parseDouble(args[i])`

Pitfall: if input is not numeric, parsing throws `NumberFormatException`.

7.5 Data types, operators, precedence, casting

7.5.1 Primitive data types (quick)

- Integer types: `byte`, `short`, `int`, `long`
- Floating types: `float`, `double`
- Other: `char` (Unicode), `boolean`

7.5.2 Operator pitfalls you must know

- **Integer division:** `5/2 == 2` (not 2.5). Use `5/2.0` or cast: `(double)5/2`.

- **Overflow:** `int` can overflow silently.

  Example: start with `int x = 2_000_000_000`; then add another billion and observe the wrap-around result.

- **Short-circuit logic:** `&&` and `||` may skip evaluating the right side.

  This is useful for null checks. First test `s != null`. Only then check whether the string length is greater than zero.

- **Precedence:** multiplication happens before addition. Use parentheses when teaching or learning.

7.5.3 Casting

Widening (safe) happens automatically: `int` → `double`. **Narrowing** may lose data and requires explicit cast: `double` → `int`.

8 Running Project Connection

Use I/O + Command Line + Types + Operators to make the first text-based version of Campus Resource Manager easier to reason about before the full object model arrives.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Mixing numeric and line-based input without understanding Scanner behavior.
- Ignoring operator precedence and misreading the result.
- Expecting Java to convert unrelated types automatically.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

10 Worked Example: Args + Scanner + Formatting

```java
import java.util.Scanner;

public class InputDemo {
```

```java
    public static void main(String[] args) {
        int maxMarks = 100;
        if (args.length > 0) maxMarks = Integer.parseInt(args[0]);

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter marks (0-" + maxMarks + "): ");
        int marks = sc.nextInt();

        double percent = marks * 100.0 / maxMarks;
        System.out.printf("Hello %s, marks=%d/%d, percent=%.2f%%%n",
               name, marks, maxMarks, percent);
        sc.close();
    }
}
```

What to observe:

- How `args` is always a string array and needs parsing.
- How we used a default value (100) and optionally override it using `args[0]`.
- How formatted output (`printf`) improves readability.
- Why we used `100.0` in the formula (to avoid integer division mistakes).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Console I/O: `Scanner` (most common for beginners)* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. What is the output of `System.out.println(5/2);`? **Answer:** `2`.

2. How to get `2.5` from 5 and 2? **Answer:** `5/2.0` or `(double)5/2`.
3. If you run `java Demo A B`, what is `args.length`? **Answer:** 2.

13 Self-Study Practice (no solutions)

1. Write a program `SumArgs` that sums integer command line arguments.
2. Write a program that reads a line and prints: (a) length, (b) first character, (c) uppercase version.
3. Create 5 expressions and predict results using precedence; then verify by running Java.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. Why is `Scanner.nextLine()` tricky after `nextInt()`?

Sample answer: `nextInt()` leaves the newline in the input buffer, so the next `nextLine()` may read the leftover newline and return an empty string. We can consume the newline by calling `nextLine()` once after reading the number.

Q2. Why is `5/2` not 2.5 in Java?

Sample answer: Both operands are integers, so Java performs integer division and discards the fractional part. If either operand is `double`, it performs floating-point division.

Q3. When should you cast explicitly in Java?

Sample answer: When Java cannot prove the conversion is safe or when narrowing may lose information.